

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	S.Mahammed Nayeem	Reg. No.:	192425294
Section / Batch:	2024 - AI &ML	Date:	23-07-2026

Instructions to Students

- Answer BOTH scenario-based questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions	Marks
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – SCENARIO BASED ASSESSMENT

Q1. A retail company maintains daily sales data that must be searched quickly to find specific transaction IDs. The dataset is unsorted due to continuous updates. The system currently uses Linear Search, but performance degrades as data grows.

- Explain how Linear Search operates in this scenario.
- Analyze the time complexity for best, average, and worst cases.
- Suggest whether sorting + Binary Search would improve performance and justify your decision.

Q2. A navigation system processes coordinates of multiple delivery points to identify the farthest pair of locations for route planning. The system currently checks all possible pairs using a brute-force approach.

- Explain how the brute-force method works for this problem.
- Derive the time complexity for computing all pair distances.
- Analyze why this approach becomes inefficient as the number of points increases.

Code of Conduct

I S.Mahammed Nayeem (192425294) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Mahammed Nayeem

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Question No.	Problem Understanding & Context Analysis (25%)	Application of Concepts (30%)	Analytical & Decision-Making Skills (20%)	Solution Design & Approach (15%)	Clarity, Justification & Presentation (10%)	Total Marks (Each – 50)
Q1	4	3	3	2	4	40
Q2	4	4	4	3	2	46
Overall Total (100)						86

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1 : Scenario-Based Execution – System Log Analyzer Using Brute-Force String Matching :

Scenario

A cybersecurity team maintains a server that generates thousands of log entries every hour. The administrator needs to search for the error message "ERROR" within a large log file to identify system failures. Since the logs are stored as plain text, the system uses the Brute-Force String Matching algorithm to locate the required pattern.

Text (Log File):

Pattern:

Execution Steps

The algorithm compares the pattern with every possible position in the text.

Step	Starting Position	Characters Compared	Result
1	SYSTEM	SYSTEM vs ERROR	Mismatch
2	YSTEMS	YSTEM vs ERROR	Mismatch
3	STEMST	STEMS vs ERROR	Mismatch
4	TEMSTA	TEMST vs ERROR	Mismatch
5	EMSTAR	EMSTA vs ERROR	Mismatch
6	MSTART	MSTAR vs ERROR	Mismatch
7	STARTE	START vs ERROR	Mismatch
8	TARTER	TARTE vs ERROR	Mismatch
9	ARTERR	ARTER vs ERROR	Mismatch
10	ERROR	ERROR = ERROR	Pattern Found

Number of Comparisons

- The pattern is found after checking 10 starting positions.
- Once all characters match, the search stops.

Time Complexity

- Best Case: $O(n)$ – Pattern found immediately.
- Average Case: $O(n \times m)$ – Pattern found after several comparisons.
- Worst Case: $O(n \times m)$ – Pattern appears at the end or is absent.

Where:

- n = Length of the log file
- m = Length of the search pattern

Verification

The pattern "ERROR" exists in the text beginning at the correct position. The algorithm successfully detects the required error message.

Interpretation

Brute-Force String Matching is simple and reliable for searching log files. However, in large system logs containing millions of characters, it performs many unnecessary comparisons, making the search slower. Therefore, modern log analysis tools prefer algorithms such as KMP, Boyer-Moore, or Rabin-Karp, which reduce redundant comparisons and improve search efficiency.

Question 2 : Programming IDE Searches for Function Names Using Brute-Force String Matching

Problem Statement

A programming IDE (Integrated Development Environment) needs to search for a function name within a large codebase. It uses the **Brute-Force String Matching** algorithm, which compares the search pattern with the source code character by character until the function name is found.

Example:

- **Text (Source Code):** `int main() { calculateSum(); return 0; }`
- **Pattern:** `calculateSum`

a) Matching Process

The Brute-Force String Matching algorithm works by comparing the pattern with every possible position in the text.

Steps

1. Start from the first character of the source code.
2. Compare the first character of the pattern with the current character in the text.
3. If the characters match, continue comparing the remaining characters.
4. If a mismatch occurs, shift the pattern one position to the right.
5. Repeat the process until the entire pattern matches or the end of the text is reached.

Example Execution

Step	Starting Position	Comparison	Result
1	int	int vs calculateSum	Mismatch
2	nt	nt vs calculateSum	Mismatch
3	t	t vs calculateSum	Mismatch
4	main	main vs calculateSum	Mismatch
5	{	{ vs calculateSum	Mismatch
6	calculateSum	calculateSum = calculateSum	Pattern Found

The IDE stops searching after finding the complete match.

b) Time Complexity Analysis

Let:

- **n** = Length of the source code
- **m** = Length of the function name

Best Case

- **$O(n)$**
- The function name is found immediately or mismatches occur at the first character.

Average Case

- **$O(n \times m)$**
- The algorithm performs several comparisons before locating the function.

Worst Case

- **$O(n \times m)$**
- The function name is at the end of the code or does not exist, requiring comparisons at every possible position.

c) Justification for Optimized Searching Algorithms

Brute-Force String Matching is easy to implement but becomes inefficient for large software projects because it repeatedly compares the same characters.

Optimized algorithms such as:

- **Knuth-Morris-Pratt (KMP)**
- **Boyer-Moore**
- **Rabin-Karp**

reduce unnecessary comparisons and improve search speed.

Advantages of Optimized Algorithms

- Faster searching in large codebases.
- Reduced execution time.
- Better IDE responsiveness.
- Efficient navigation across thousands of source files.
- Improved productivity for developers.